Managing Application Stacks with Docker Compose Commands

In this train, we'll combine services, networks, and volumes into a cohesive docker-compose.yml file, ready for deployment. Through practical exercises, we'll integrate all these components to orchestrate a complete application stack.

Learning objectives

> - Be able to combine services, networks, and volumes into a cohesive docker-compose.yml file, ready for deployment.

Managing Application Stacks with Docker Compose

Once an application stack is defined in a `docker-compose.yml` file, you can manage it using Docker Compose commands. Below are essential commands for application stack management:

More information on Docker Compose: [click here](https://docs.docker.com/compose/).

> **Prerequisites:**
> - Understanding of Docker Compose YAML configuration files.
> - Experience with defining services, networks, and volumes in Docker Compose.

1. Starting and Stopping Containers

> - To start all services defined in the `docker-compose.yml` file:
> `docker-compose up -d`
>
> > **Explanation:**
> > - `-d` flag runs the containers in detached mode, allowing them to run in the background.

- To stop and remove all running services and networks:
  `docker-compose down`

2. Checking Running Services

- To list all active services in your stack:
  `docker-compose ps`

3. Viewing and Monitoring Logs

- To check real-time logs of running services:
  `docker-compose logs -f`

> **Explanation:**
> - `-f` flag ensures continuous log output as services generate logs.

4. Restarting Services

- To restart all services:
  `docker-compose restart`
- To restart a specific service:
  `docker-compose restart app`

5. Scaling Services

- If your application stack needs to scale, use the following command:
  `docker-compose up --scale app=3 -d`

> **Note:**
> - scales the `app` service to three instances.

6. Running One-Time Commands Inside a Service

- To run a shell command inside a running service container:
  `docker-compose exec app ls /usr/src/app`

  > **Explanation:**
  > - executes `ls` inside the `app` service container.

7. Updating Services and Rebuilding Images

- To rebuild service images without starting containers:
  `docker-compose build`
- To pull the latest images and restart the services:
  `docker-compose pull && docker-compose up -d`

8. Cleaning Up Unused Containers and Resources

- To remove stopped services:
  `docker-compose rm -f`
- To remove orphaned volumes created by Docker Compose:
  `docker volume prune`

Best Practices

- Use `docker-compose up -d` for background execution and `docker-compose down` for clean shutdown.
- Monitor logs regularly using `docker-compose logs -f` to detect issues early.
- Scale services dynamically when needed using `docker-compose up --scale`.
- Regularly update and rebuild images with `docker-compose pull` and `docker-compose build`.

Conclusion

Docker Compose provides a powerful way to manage multi-container applications. By mastering these commands, you can effectively control, monitor, scale, and troubleshoot your application stacks.

Key Takeaways

- **Centralized Management:** Docker Compose simplifies the definition and management of multi-container applications through a single `docker-compose.yml` file.
- **Essential Commands:** Commands like `docker-compose up`, `down`, `ps`, `logs`, `restart`, `scale`, `exec`, `build`, `pull`, and `rm` are fundamental for controlling the lifecycle of your application stack.
- **Background Execution & Clean Shutdown:** Use `docker-compose up -d` to run services in the background and `docker-compose down` for a graceful stop and removal of containers and networks.
- **Monitoring & Troubleshooting:** The `docker-compose logs -f` command is crucial for real-time monitoring and debugging by streaming logs from your services.
- **Scalability & Updates:** Docker Compose enables easy scaling of services (e.g., `--scale`) and efficient updating/rebuilding of images (`build`, `pull`) to keep your applications current.
- **Resource Cleanup:** Commands like `docker-compose rm` and `docker volume prune` are important for cleaning up stopped containers and unused volumes, maintaining a tidy environment.